

TD : La persistance des données en PHP

Table des matières

1) Problématique.....	1
2) Exercices	1
3) Conclusion.....	14

1) Problématique

Nous souhaitons apprendre à rendre les objets persistants, en les sauvegardant dans une base de données. Pour cela nous allons effectuer plusieurs exercices en rapport avec ce sujet en s'aidant d'outils comme Xampp et Notepad++.

2) Exercices

Exercice 1 :

1) Accès a phpmyadmin

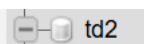
Nous devons d'abord accéder à phpmyadmin en passant par Xampp, nous lançons donc le logiciel puis on active Apache et MySQL :

Service	Module	PID(s)	Port(s)	Actions
☐	Apache	38632 34712	80, 443	Stop Admin Config
☐	MySQL	36488	3306	Stop Admin Config

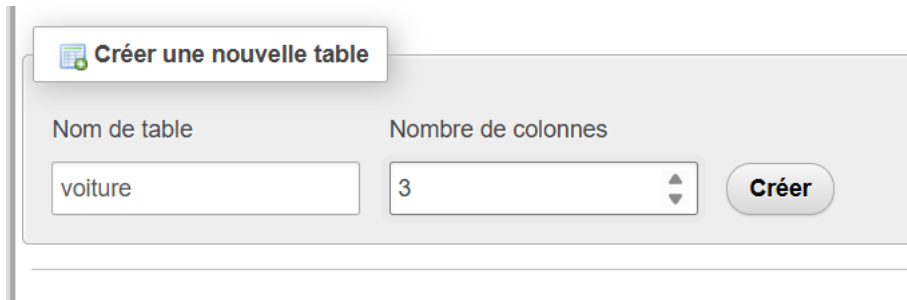
Lorsque c'est fait on clique sur « **Admin** » afin de pouvoir accéder à l'interface phpmyadmin.

2) Création de table

Dans phpmyadmin, nous allons commencer par créer une base de données « TD2 » :



Ensuite on crée une table voiture avec 3 colonnes en remplissant les champs et en cliquant sur « **Créer** » :



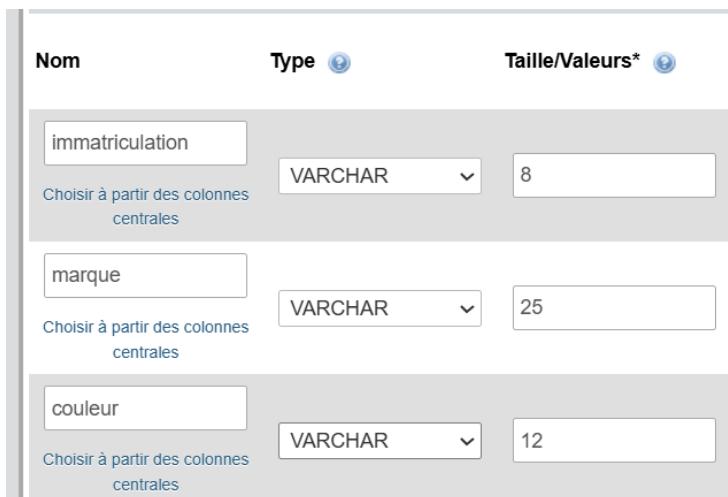
Créer une nouvelle table

Nom de table: voiture

Nombre de colonnes: 3

Créer

Ensuite on ajoute 3 attributs : **immatriculation**, **marque**, **couleur**. Chaque attribut est de type VARCHAR et leur taille sont respectivement 8, 25 et 12 :



Nom	Type	Taille/Valeurs*
immatriculation Choisir à partir des colonnes centrales	VARCHAR	8
marque Choisir à partir des colonnes centrales	VARCHAR	25
couleur Choisir à partir des colonnes centrales	VARCHAR	12

On peut par la suite cliquer sur « **Enregistrer** » :



Enregistrer

3) Insertion de données

Après avoir enregistré on se retrouve sur le menu de la table voiture, à partir d'ici nous allons cliquer sur l'onglet « **Insérer** » :



Insérer

Dans les valeurs on peut choisir les données à insérer, je vais ajouter une Toyota rouge qui a pour immatriculation AA-000-AA (sans tirets) :

Colonne	Type	Fonction	Null	Valeur
immatriculation	varchar(8)			AA000AA
marque	varchar(25)			Toyota
couleur	varchar(12)			Rouge

Exécuter

On peut ensuite cliquer sur **Exécuter** et cela va créer l'instance que nous avons configuré :

immatriculation	marque	couleur
AA000AA	Toyota	Rouge

Exercice 2 :

1) Création de conf.php

Deux questions nous est posé :

Où doit on enregistrer une page web ?

Pour enregistrer une page web il suffit d'enregistrer le fichier de celle-ci dans le document **htdocs** de XAMPP.

Qu'est-ce qu'un attribut ou une méthode classique ?

Un attribut ou une méthode statique est un champ/une fonction rattaché directement à la classe et non à un objet.

Nous allons maintenant créer un fichier Conf.php dans le document htdocs de XAMPP.

On utilise l'outil Notepad++ pour modifier le fichier (téléchargeable ici : <https://notepad-plus-plus.org/downloads/>)

Ensuite on colle le code qui nous est donné et on remplace les « à remplir » (j'ai créé une base « root » pour l'occasion) :

```

<?php
class Conf {

    static private $databases = array(
        // ou localhost sur votre machine
        'hostname' => 'localhost',
        // A l'IUT, vous avez une BDD nommée comme votre login
        // Sur votre machine, vous devrez créer une BDD
        'database' => 'root',
        // A l'IUT, c'est votre login
        // Sur votre machine, vous avez sûrement un compte 'root'
        'login' => 'root',
        // A l'IUT, c'est votre mdp (INE par défaut)
        // Sur votre machine personnelle, vous avez créé ce mdp à l'installation
        'password' => ''
    );

    static public function getLogin() {
        // en PHP l'indice d'un tableau n'est pas forcément un chiffre.
        return self::$databases['login'];
    }
}
?>

```

3. Cette fois-ci nous allons créer un fichier pour tester Conf.php, son nom sera « **testconf.php** ».

La bonne URL est celle qui passe par le serveur Apache et non par notre PC.

Dans testconf.php on copie-collé le code donné :

```

<?php
// On inclut les fichiers de classe PHP avec require_once
// pour éviter qu'ils soient inclus plusieurs fois
require_once 'Conf.php';
// On affiche le login de la base de données
echo Conf::getLogin();
?>

```

4. On va ajouter 3 fonctions dans Conf.php et les tester avec testconf.php (qu'on va aussi devoir modifier) :

```

} <?php
class Conf {

    static private $databases = array(
        'hostname' => 'localhost',
        'database' => 'td2',
        'login' => 'root',
        'password' => ''
    );

    static public function getLogin() {
        return self::$databases['login'];
    }

    static public function getHostname() {
        return self::$databases['hostname'];
    }

    static public function getDatabase() {
        return self::$databases['database'];
    }

    static public function getPassword() {
        return self::$databases['password'];
    }
}
-?>

```

Ici on ajoute les fonctions `getHostname`, `getDatabase` et `getPassword`. On garde la même architecture que `getLogin` pour les créer.

Dans `testconf.php`, on test les commandes de cette façon :

```
<?php
require_once 'Conf.php';

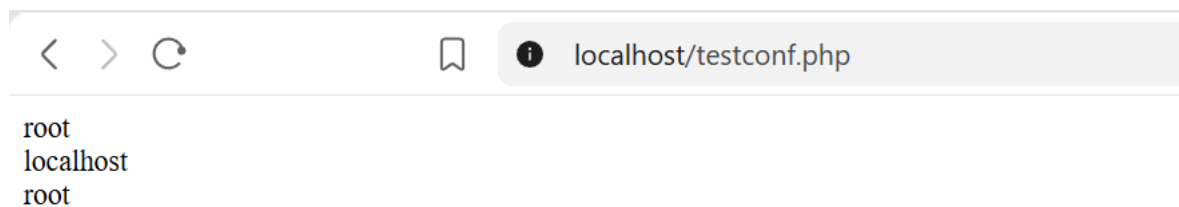
echo Conf::getLogin();
echo "<br>";

echo Conf::getHostname();
echo "<br>";

echo Conf::getDatabase();
echo "<br>";

echo Conf::getPassword();
?>
```

Résultat :



localhost/testconf.php

root
localhost
root

Exercice 3 : Enregistrement du projet avec Git

L'exercice est différent car le TD1 a été fait y'a plusieurs mois donc j'ai improvisé

On va simplement ici créer un dépôt et ajouter un commit.

On utilise donc la commande **git init** sur le dossier qui nous intéresse (ici `htdocs`) :

```
C:\xampp\htdocs>git init
Initialized empty Git repository in C:/xampp/htdocs/.git/

C:\xampp\htdocs>
```

Ensuite on fait **git add** . pour ajouter tous les fichiers présents à notre dépôt puis on crée un commit pour sauvegarder l'état actuel du dossier :

```
C:\xampp\htdocs>git add .
warning: in the working copy of 'applications.html', LF will be replaced by CRLF the
warning: in the working copy of 'bitnami.css', LF will be replaced by CRLF the next t
warning: in the working copy of 'dashboard/404.html', LF will be replaced by CRLF the
warning: in the working copy of 'dashboard/de/faq.html', LF will be replaced by CRLF
warning: in the working copy of 'dashboard/de/howto.html', LF will be replaced by CRL
warning: in the working copy of 'dashboard/de/howto_platform_links.html', LF will be
touches it
warning: in the working copy of 'dashboard/de/howto_shared_links.html', LF will be re
uches it
warning: in the working copy of 'dashboard/de/index.html', LF will be replaced by CRLF
```

```
C:\xampp\htdocs>git commit -m "TD2, exercice 3"
[master (root-commit) 5bd453e] TD2, exercice 3
 346 files changed, 77278 insertions(+)
 create mode 100644 Conf.php
 create mode 100644 applications.html
 create mode 100644 bitnami.css
 create mode 100644 dashboard/404.html
 create mode 100644 dashboard/de/faq.html
 create mode 100644 dashboard/de/howto.html
 create mode 100644 dashboard/de/howto_platform_links.html
 create mode 100644 dashboard/de/howto_shared_links.html
 create mode 100644 dashboard/de/index.html
```

Enfin on vérifie que le commit a bien marché avec **git log** :

```
C:\xampp\htdocs>git log
commit 5bd453e67404730805ff8fca5f0113efb54bec93 (HEAD -> master)
Author: Ewan <ealexandre2804@gmail.com>
Date: Thu Sep 10 14:27:16 2026 +0200

    TD2, exercice 3
```

Exercice 4 :

1) Création de model.php

Nous allons cette fois-ci créer un projet nommé **model.php** dans lequel nous allons déclarer une classe **Model** qui possèdera un attribut statique **\$pdo** et une fonction **public static function Init()**

Voici la base :

```
<?php
class Model {
    public static $pdo
    public static function Init() {
    }
}
```

2) Initialisation de \$pdo

Pour initialiser le \$pdo on va ajouter un require_once pour que model.php puisse savoir où se trouve Conf.php :

```
<?php
require_once 'Conf.php';
class Model {
    public static $pdo
    public static function Init() {
    }
}
```

i et iii) Ensuite dans la fonction Init() on crée une nouvelle connexion grâce à la classe PHP PDO que l'on va stocker dans la variable \$pdo pour pouvoir réutiliser autant de fois que l'on souhaite cette connexion, on précise à notre code que \$pdo appartient à la classe Model avec self :

```
<?php
require_once 'Conf.php';

class Model
{
    public static $pdo;

    public static function Init()
    {
        self::$pdo = new PDO("mysql:host=$hostname;dbname=$database_name",$login,$password);
    }
}

?>
```

Le require_once servira à récupérer les variables hostname, ... directement dans le Conf.php.

ii) On va préciser au code les variables de Conf.php grâce aux getters de Conf.php (il ne faut pas oublier de les déclarer avant de faire la connexion) :

```

<?php

require_once 'Conf.php';

class Model
{
    public static $pdo;

    public static function Init()
    {
        $hostname = Conf::getHostname();
        $database_name = Conf::getDatabase();
        $login = Conf::getLogin();
        $password = Conf::getPassword();
        self::$pdo = new PDO("mysql:host=$hostname;dbname=$database_name",$login,$password);
    }
}

```

iv) Ici il faut simplement **Model::Init()**; après la fin de la classe :

```

<?php

require_once 'Conf.php';

class Model
{
    public static $pdo;

    public static function Init()
    {
        $hostname = Conf::getHostname();
        $database_name = Conf::getDatabase();
        $login = Conf::getLogin();
        $password = Conf::getPassword();
        self::$pdo = new PDO("mysql:host=$hostname;dbname=$database_name",$login,$password);
    }
}

Model::Init();

```

v) Maintenant on crée un fichier testModel.php, puis on y colle le code donné :

```

1 <?php
2
3 require_once "Model.php";
4
5 echo "Connexion réussie !";
6
7

```

Ensuite on accède au navigateur pour tester :

localhost/testModel.php

Connexion réussie !

Exercice 5 :

1) Exception PDO

Nous allons ajouter un try catch pour récupérer les messages d'erreurs s'il y a un problème de connexion. Pour cela on ajoute notre connexion dans le **try {** puis on ajoute un **catch {** qui servira à récupérer la classe d'exception **PDOException** que l'on stocke dans une variable **\$e** et que l'on associera à la fonction déjà intégré à PHP **getMessage()** et que l'on affichera avec un **echo :**

```
<?php
require_once 'Conf.php';

class Model
{
    public static $pdo;

    public static function Init()
    {
        $hostname = Conf::getHostname();
        $database_name = Conf::getDatabase();
        $login = Conf::getLogin();
        $password = Conf::getPassword();

        try {
            self::$pdo = new PDO("mysql:host=$hostname;dbname=$database_name", $login, $password);
        }
        catch(PDOException $e) {
            echo $e->getMessage();
            die();
        }
    }
}

Model::Init();
```

2) UTF-8 et gestion des erreurs

Ici on modifie simplement le code par ce qui nous est donné :

```
<?php
require_once 'Conf.php';

class Model
{
    public static $pdo;

    public static function Init()
    {
        $hostname = Conf::getHostname();
        $database_name = Conf::getDatabase();
        $login = Conf::getLogin();
        $password = Conf::getPassword();

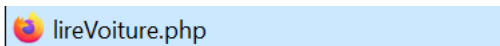
        try {
            self::$pdo = new PDO("mysql:host=$hostname;dbname=$database_name", $login, $password, array(PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8"));
            self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        }
        catch(PDOException $e) {
            echo $e->getMessage();
            die();
        }
    }
}

Model::Init();
?>
```

Exercice 6 :

1) Création de lireVoiture.php

On crée avec Notepad++ un fichier **lireVoiture.php** :



2) Inclusion de model.php

Pour inclure model.php on utilise le **require once** :

```
<?php
require_once "Model.php";
?>
```

3) Fonction query

On va appeler la fonction **query** et la stocker dans une variable **\$rep** comme ceci :

```
<?php
require_once "Model.php";

$rep = Model::$pdo->query("SELECT * FROM voiture");
?>
```

4) Fonction fetchAll

Maintenant on utilise la fonction **fetchAll** pour récupérer la réponse présente dans la variable **\$rep** et on stocke le tableau dans une variable **\$tab_obj** (on y inclus le style de fetch) :

```
<?php
require_once "Model.php";

$rep = Model::$pdo->query("SELECT * FROM voiture");

$tab_obj = $rep->fetchAll(PDO::FETCH_OBJ);
?>
```

Ensuite on utilise une boucle pour parcourir le tableau et afficher les voitures avec un **for each** + **echo** :

```
<?php
require_once "Model.php";

$rep = Model::$pdo->query("SELECT * FROM voiture");

$tab_obj = $rep->fetchAll(PDO::FETCH_OBJ);

foreach ($tab_obj as $obj) {
    echo $obj->immatriculation . " " . $obj->couleur . " " . $obj->marque . "<br>";
}
?>
```

Exercice 7 :

1) Inclure le fichier de la classe voiture

Pour faire cela, rien de plus simple il suffit de faire un require once :

```
<?php  
require_once "Model.php";  
require_once "Voiture.php";
```

2) Adaptation du constructeur de Voiture

Ici on nous devons avoir créé un fichier Voiture.php mais étant donné que le TD1 a déjà été fait il y a longtemps je n'y ait plus accès. Je recrée donc un fichier Voiture.php

Dans le nouveau Voiture.php, le constructeur est à jour grâce au code donné sur le pdf :

```

<?php
class Voiture {
    private $immatriculation;
    private $marque;
    private $couleur;

    public function __construct($m = NULL, $c = NULL, $i = NULL) {
        if (!is_null($m) && !is_null($c) && !is_null($i)) {
            $this->marque = $m;
            $this->couleur = $c;
            $this->immatriculation = $i;
        }
    }

    public function getImmatriculation() {
        return $this->immatriculation;
    }

    public function getMarque() {
        return $this->marque;
    }

    public function getCouleur() {
        return $this->couleur;
    }

    public function setImmatriculation($immatriculation) {
        $this->immatriculation = $immatriculation;
    }

    public function setMarque($marque) {
        $this->marque = $marque;
    }

    public function setCouleur($couleur) {
        $this->couleur = $couleur;
    }

    public function afficher() {
        echo "Voiture " . $this->immatriculation . " de marque " . $this->marque . " (couleur : " . $this->couleur . ")<br>";
    }
}
?>

```

3) fetchAll() dans lireVoiture.php

Maintenant nous pouvons enfin fetchAll() pour convertir tout le SQL en PHP et afficher le tableau grâce à **foreach** :

```

<?php
require_once "Model.php";
require_once "Voiture.php";

$rep = Model::$pdo->query("SELECT * FROM voiture");

$rep->setFetchMode(PDO::FETCH_CLASS, 'Voiture');
$stab_voit = $rep->fetchAll();

foreach ($stab_voit as $v) {
    $v->afficher();
}
?>

```

Exercice 8 : Création de la fonction statique getAllVoitures()

Dans voiture.php on ajoute la fonction statique :

```
public static function getAllVoitures() {  
    $rep = Model::$pdo->query("SELECT * FROM voiture");  
    $rep->setFetchMode(PDO::FETCH_CLASS, 'Voiture');  
    $tab_voit = $rep->fetchAll();  
    return $tab_voit;  
}
```

Puis on adapte lireVoiture.php :

```
<?php  
require_once "Model.php";  
require_once "Voiture.php";  
  
$tab_voit = Voiture::getAllVoitures();  
  
foreach ($tab_voit as $v) {  
    $v->afficher();  
}
```

Résultat :

< > ↻ ⓘ localhost/lireVoiture.php

Voiture AA000AA de marque Toyota (couleur : Rouge)

Exercice 9 : Ajout de debug

Dans conf.php on ajoute l'attribut debug et son getter publique :

```
static private $debug = True;  
  
static public function getDebug() {  
    return self::$debug;  
}
```

Ensuite dans Model.php on fait un try { catch { pour l'affichage des messages d'erreurs

```
<?php
require_once 'Conf.php';

class Model
{
    public static $pdo;

    public static function Init()
    {
        $hostname = Conf::getHostname();
        $database_name = Conf::getDatabase();
        $login = Conf::getLogin();
        $password = Conf::getPassword();

        try {
            self::$pdo = new PDO("mysql:host=$hostname;dbname=$database_name", $login, $password, array(PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8"));
            self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        }
        catch(PDOException $e) {
            if (Conf::getDebug()) {
                echo $e->getMessage(); // affiche un message d'erreur
            } else {
                echo 'Une erreur est survenue <a href=""> retour a la page d\'accueil </a>';
            }
            die();
        }
    }
}

Model::Init();
?>
```

Il y a une partie 3 au TP mais ça nous demande encore d'avoir effectué le TP1, j'ai donc décidé de ne pas le faire car je n'ai pas les fichiers nécessaires.

3) Conclusion

En conclusion, j'ai réussi à rendre les objets persistants en faisant une liaison entre MySQL et PHP. Grâce à PDO et au mode FETCH_CLASS, les données de la base sont désormais directement converties en instances de la classe Voiture pour être manipulées et affichées en programmation orientée objet.